MODULE 07

RAG vs LightRAG vs Wiki-RAG

세 가지 검색 증강 생성 아키텍처의 원리, 구현, 그리고 성능 비교
Andrej Karpathy의 LLM-Wiki 패턴 (2026.04) 포함

3	5	4	7.8
ARCHITECTURES	NBER PAPERS	NOTEBOOKS	BEST SCORE
RAG · LightRAG · Wiki	실제 논문 벤치마크	실습 Lab 포함	Wiki-RAG (10점 만점)

> *"RAG는 매 질문마다 지식을 재발견한다. Wiki는 지식을 한 번 컴파일하고 계속 축적한다."*
>
> *— Andrej Karpathy, "LLM Knowledge Bases", April 2026*

SLIDE 01 · PROBLEM

LLM의 근본 문제: 지식의 한계

LIMITATION 01

Knowledge Cutoff

훈련 데이터 이후의 정보를 모른다. 2024년 논문? 오늘의 주가? → 답변 불가

LIMITATION 02

Hallucination

존재하지 않는 논문을 인용하고, 그럴듯한 거짓을 생성한다. 연구에서는 치명적.

LIMITATION 03

Context Window

논문 5편 = ~300페이지. 모든 걸 프롬프트에 넣을 수 없다. 어떤 부분을 선택할 것인가?

SOLUTION

RAG: Retrieval-Augmented Generation

외부 지식 소스에서 **관련 정보를 검색**하여 LLM 프롬프트에 주입

- **Knowledge cutoff** → 실시간 DB에서 검색
- **Hallucination** → 출처 기반 답변 강제
- **Context limit** → 관련 부분만 선택 검색

핵심 질문: "검색"을 어떻게 하느냐에 따라 RAG의 품질이 결정된다. 오늘 세 가지 접근법을 비교한다.

Standard RAG: Chunk → Embed → Retrieve

INPUT
문서들
PDF, 논문, 보고서

→

STEP 1
청킹
2000자 단위로 분할

→

STEP 2
임베딩
텍스트 → 384차원 벡터

→

STEP 3
FAISS 인덱스
벡터 저장 & 검색

→

QUERY TIME
검색 + LLM
Top-K 청크 → 답변

STRENGTHS

장점

- **구축 3초** — 임베딩만 하면 끝
- **확장성** — 수백만 문서 처리 가능
- **단순함** — 이해하기 쉬운 파이프라인
- **성숙한 생태계** — LangChain, LlamaIndex

WEAKNESSES

약점

- **문맥 손실** — 청크가 원문의 맥락을 잃음
- **무상태** — 매 쿼리마다 지식을 재발견
- **크로스 문서 약함** — 논문 간 연결 못함
- **불투명** — 왜 이 청크가 검색됐는지?

```
# Python 핵심 코드 (3줄 요약)
chunks = chunk_text(paper, size=2000)
index  = FAISS.from_vectors( engine.encode (chunks))
answer = LLM(question + index. search (question, top_k=5))
```

LightRAG: Knowledge Graph + Vector Search

INPUT
문서들
PDF, 논문

→

STEP 1
LLM 추출
엔티티 & 관계

→

STEP 2
지식 그래프
노드 + 엣지

→

QUERY TIME
그래프 탐색
1-hop 이웃 확장

+

결합
벡터 검색
Graph + Vector

KEY INNOVATION

엔티티 + 관계 추출

LLM이 논문에서 구조화된 지식을 추출합니다:

```
Entity: Card (person)
Entity: Human Capital (concept)
Relation: Card →studies→ Human Capital
Relation: Card →uses→ Census Data
```

DUAL RETRIEVAL

그래프 + 벡터 결합

질문에서 엔티티를 찾고, 그래프를 탐색하여 **관련 엔티티의 이웃**까지 확장합니다.

- **벡터 검색**: 의미적 유사도
- **그래프 탐색**: 구조적 관계
- **결합**: 두 신호를 합쳐서 LLM에 전달

핵심 차이: Standard RAG는 "비슷한 텍스트"를 찾지만, LightRAG는 "관련된 엔티티와 그 관계"도 함께 제공한다. 크로스 논문 질문에서 특히 효과적.

Karpathy's LLM-Wiki: Compile, Don't Retrieve

> *"대부분의 RAG 경험은 이렇다: 파일을 올리면 LLM이 질문할 때마다 관련 조각을 찾는다. 이것은 작동하지만, LLM은 매 질문마다 처음부터 지식을 재발견하고 있다. 축적이 없다."*
>
> *— Andrej Karpathy, April 4, 2026 · GitHub Gist (4,600+ stars in 2 days)*

LAYER 1

raw/ (원본)

불변의 소스 문서. LLM은 읽기만 하고 **절대 수정하지 않는다**.

논문, 보고서, 데이터 파일

LAYER 2

wiki/ (컴파일된 지식)

LLM이 **생성하고 관리**하는 마크다운 위키. 요약, 개념 페이지, 크로스 레퍼런스.

index.md + 논문 요약 + 개념 페이지

LAYER 3

schema (규칙)

위키의 **구조와 규칙**을 정의. LLM에게 "어떻게 위키를 관리할지" 지시.

CLAUDE.md / AGENTS.md

패러다임 전환: 임베딩도, 벡터 DB도 없다. 그냥 **마크다운 파일**이다. 사람이 직접 읽고, 편집하고, Git으로 버전 관리할 수 있다. 완전한 투명성.

Wiki-RAG의 4가지 핵심 연산

OPERATION 01

📄 Ingest (수집)

새 문서를 위키에 추가:

- 원본 → **raw/** 에 저장
- LLM이 읽고 **요약 페이지** 생성
- 관련 **10-15개 기존 페이지** 업데이트
- **index.md** 갱신, **log.md** 기록

OPERATION 02

🔍 Query (질의)

위키를 탐색하여 답변:

- **index.md** 읽기 → 관련 페이지 식별
- 관련 위키 **페이지 읽기**
- 여러 페이지 **종합하여 답변** 생성
- 좋은 답변은 **새 위키 페이지**로 저장

OPERATION 03

🔧 Lint (건강 검사)

위키 품질 주기적 점검:

- **고아 페이지** 발견 (아무도 링크 안 함)
- **깨진 링크** 수리 ([[없는페이지]])
- 페이지 간 **모순** 감지 & 해결
- **데이터 갭** 식별 & 조사 목록 생성

OPERATION 04

📋 Index & Log (목록 & 기록)

위키의 네비게이션 시스템:

- **index.md**: 모든 페이지의 한 줄 요약
- **log.md**: 시간순 작업 기록
- LLM의 **진입점**: index → 관련 페이지
- 사람도 Obsidian에서 **그래프 뷰**로 탐색

Karpathy의 통찰: "지식 기반 유지보수의 지루한 부분은 읽기나 생각이 아니라 — 장부 정리다. LLM은 지루해하지 않고, 크로스 레퍼런스 업데이트를 잊지 않고, 한 번에 15개 파일을 수정할 수 있다."

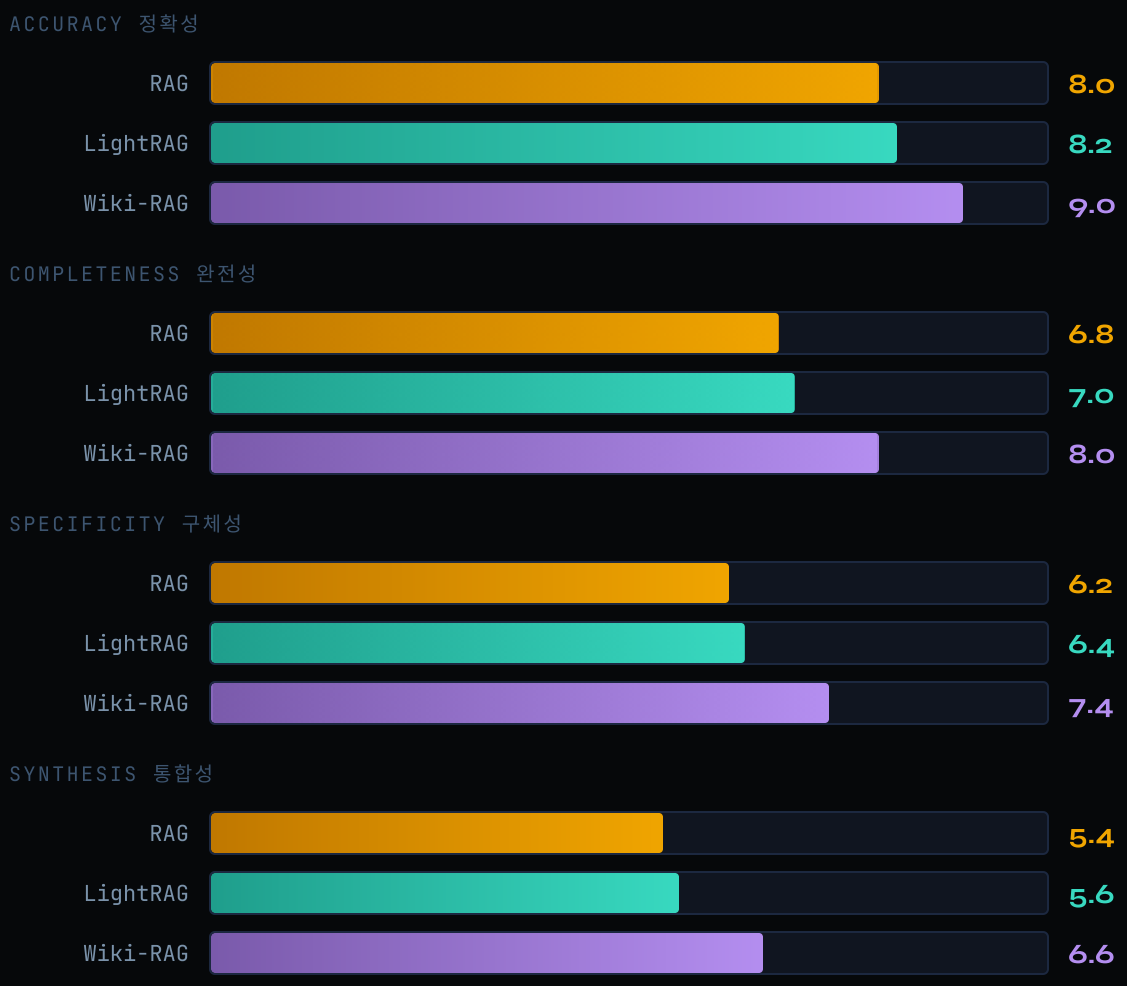
SLIDE 06 · COMPARISON

아키텍처 비교

관점	STANDARD RAG	LIGHTRAG	KARPATHY WIKI-RAG
저장소	벡터 DB (FAISS, Chroma)	벡터 DB + 지식 그래프	마크다운 파일 📁
검색 방식	임베딩 유사도	그래프 탐색 + 벡터	index.md → 페이지 탐색
지식 상태	무상태 (매번 재발견)	반영구 그래프	영구, 축적됨
투명성	불투명 (벡터)	부분 투명	완전 투명 (.md)
크로스 레퍼런스	없음	엔티티 그래프	[[wiki links]]
초기 비용	낮음 (3초)	중간 (56초)	높음 (72초)
확장성	수백만 문서	수십만 문서	수백 문서
인간 협업	어려움	그래프 시각화 가능	직접 편집 가능

벤치마크 결과: 5편 NBER 논문, 5개 질문

LLM-as-Judge (gpt-4o-mini) 평가 · 각 차원 1-10점



6.6
STANDARD RAG
전체 평균

6.8
LIGHTRAG
+3% vs RAG

7.8
WIKI-RAG
+18% vs RAG

Wiki-RAG 우위: 통합성(Synthesis)에서 가장 큰 격차. 사전 컴파일된 크로스 레퍼런스가 핵심.

질문 유형별 성능 분석

FACTUAL · EASY

단일 논문 팩트 검색

"Card et al.의 주요 발견은?"

7.0 · 7.0 · 7.5

RAG · LightRAG · Wiki

세 방법 모두 비슷. 단일 논문 검색은 Standard RAG로도 충분.

SYNTHESIS · HARD

크로스 논문 종합

"고용주 신용조회와 정보 비대칭의 관계는?"

5.9 · 7.2 · 7.8

RAG · LightRAG · Wiki

RAG 최약: 5.2점 (Q4). 관련 없는 청크 검색. Wiki는 이미 개념 페이지에 연결 정보 보유.

AGGREGATION · MEDIUM

전체 논문 집계

"모든 논문의 데이터셋과 방법론은?"

7.2 · 5.5 · 8.2

RAG · LightRAG · Wiki

LightRAG 최약: 그래프가 데이터셋 메타데이터를 잘 캡처하지 못함. Wiki는 각 요약 페이지에 이미 정리됨.

핵심 교훈: 어떤 방법이 "최고"인가? → **질문 유형에 따라 다르다.** 팩트 검색은 RAG, 크로스 종합은 Wiki, 관계 추론은 LightRAG. 실전에서는 **하이브리드**가 답이다.

비용과 Trade-off

지표	STANDARD RAG	LIGHTRAG	WIKI-RAG
구축 시간	3초	56초	72초
평균 쿼리 시간	5.0초	6.2초	12.7초
구축 시 LLM 호출	0회	논문당 1회	논문당 1회 + 개념 추출
쿼리 시 LLM 호출	1회 (생성만)	1회 (생성만)	2회 (탐색 + 생성)
API 비용 (전체 실습)	~$0.10	~$0.30	~$0.50
인프라 필요	FAISS/Chroma + 임베딩 모델	FAISS + LLM API	마크다운 파일만

RAG가 적합한 경우

- **대규모 문서** (10K+ 문서)
- **빠른 응답** 필요
- **단순 팩트** 질문 위주
- 실시간 업데이트 빈번

LightRAG가 적합한 경우

- **엔티티 관계**가 중요
- **크로스 문서** 질문 많음
- 중간 규모 (1K-100K)
- 그래프 시각화 활용

Wiki-RAG가 적합한 경우

- **깊은 연구** (50-200 소스)
- **지식 축적** 필요
- 인간과 **협업**
- 투명성·감사 가능성 필요

실전: 하이브리드 접근법

실제 연구에서는 세 가지를 **계층적으로 결합**합니다

L1

Standard RAG — 광범위 검색 (넓은 그물)

25,000편 논문 전체에서 벡터 유사도로 검색. "이 주제에 대한 논문을 찾아줘"

▼ 상위 50-100편 선별

L2

LightRAG — 관계 분석 (구조화)

선별된 논문에서 엔티티 추출 + 지식 그래프 구축. 저자, 방법론, 데이터셋 간 관계 매핑.

▼ 핵심 연구 주제로 압축

L3

Karpathy Wiki — 깊은 지식 축적 (연구 위키)

핵심 50편을 위키로 컴파일. 크로스 레퍼런스, 모순 탐지, 개념 종합. 연구자가 직접 편집.

실습: rag-lab 패키지

```
# 1. 클론
$ git clone <repo-url>
$ cd rag-lab

# 2. 환경 설정
$ pip install -r requirements.txt
$ cp .env.example .env
# .env에 OPENAI_API_KEY 입력

# 3. 노트북 실행
$ jupyter notebook notebooks/
```

API 비용 안내: gpt-4o-mini 기준 전체 실습 1회 ~$0.50. 캐시 사용 시 재실행 비용 $0.

LAB 01 · 30분

Standard RAG

청킹 실험, 임베딩 시각화, FAISS 검색, 청크 크기 비교

LAB 02 · 30분

LightRAG

엔티티 추출, 지식 그래프 탐색, 그래프+벡터 결합

LAB 03 · 40분

Karpathy Wiki-RAG

위키 컴파일, index 탐색, lint 검사, 증분 업데이트

LAB 04 · 20분

종합 벤치마크

3가지 방법 동시 비교, LLM-as-Judge 평가, 질문 유형별 분석

핵심 정리

Standard RAG

검색

"비슷한 텍스트 조각을
벡터로 찾는다"

빠름 | 대규모

LightRAG

연결

"엔티티와 관계를
그래프로 탐색한다"

관계 | 구조

Karpathy Wiki-RAG

컴파일

"지식을 한 번 정리하고
계속 축적한다"

투명 | 축적

> *"최고의 RAG 시스템은 세 가지를 계층적으로 결합한다: 넓은 검색(RAG), 구조적 연결(Graph), 깊은 축적(Wiki)."*
>
> *— Hybrid RAG Architecture Principle*

GITHUB REPO
rag-lab/ 클론 후 실습

REFERENCE
Karpathy Gist (2026.04)

NEXT CLASS
실습 결과 발표 & 토론